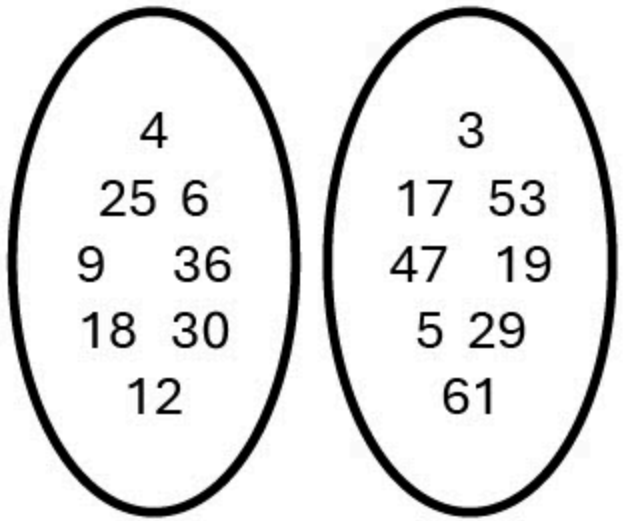
Minimum Spanning Trees

Welcome back to CS 2100!

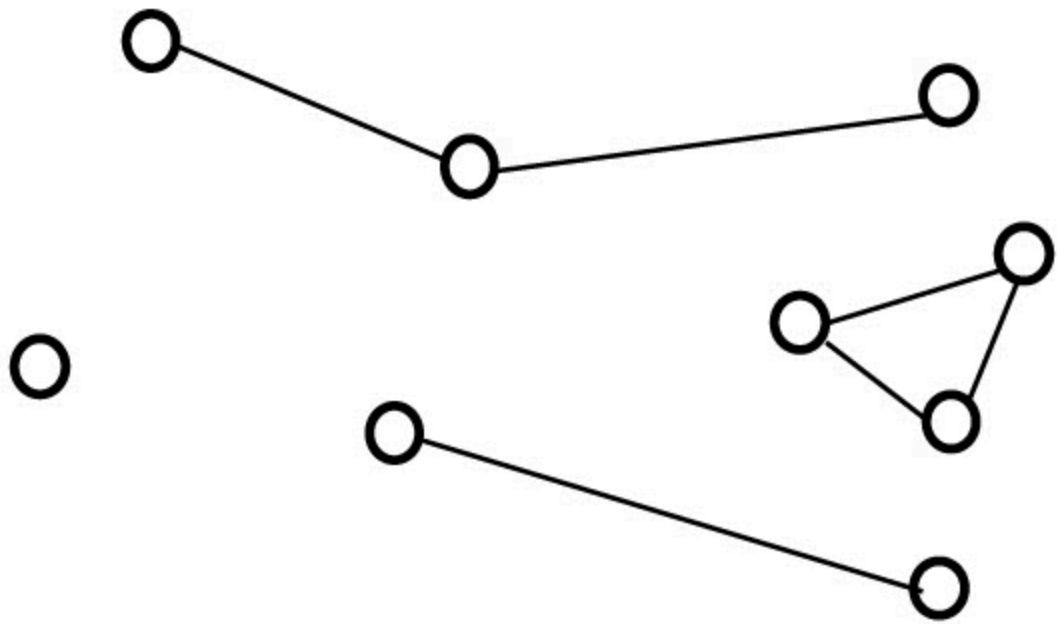
Prof. Rasika Bhalerao

Review: The Union-Find algorithm finds disjoint sets.

Disjoint sets are sets that have no elements in common.

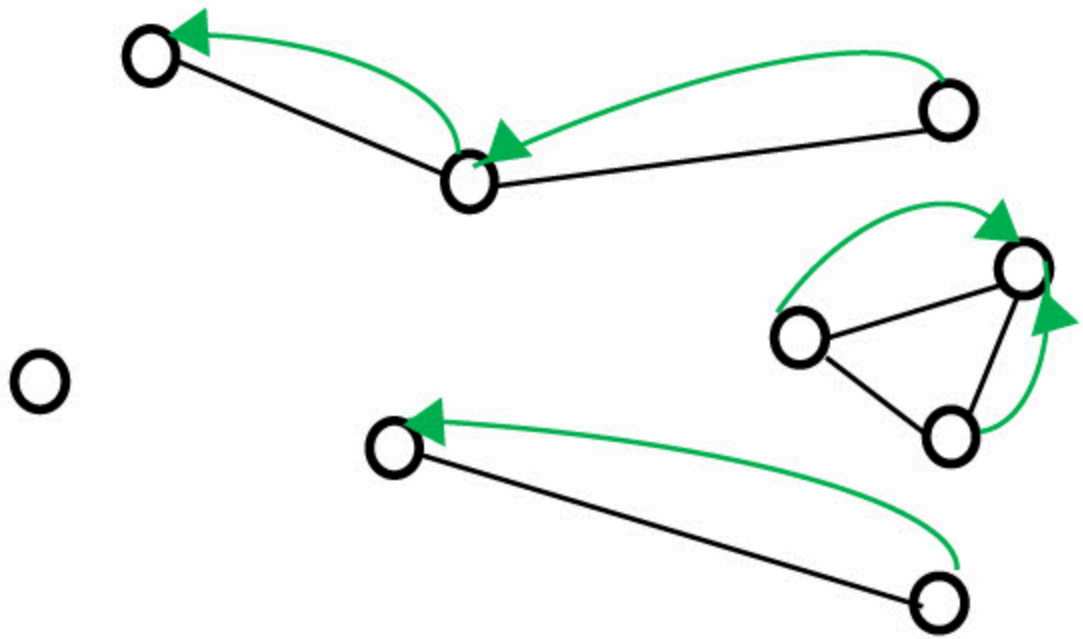


https://www.worldatlas.com/geography/continents-by-number-of-countries.html



Given: a bunch of items, and some requirements (edges) for which pairs of items need to be in the same set

Goal: put the items into disjoint sets



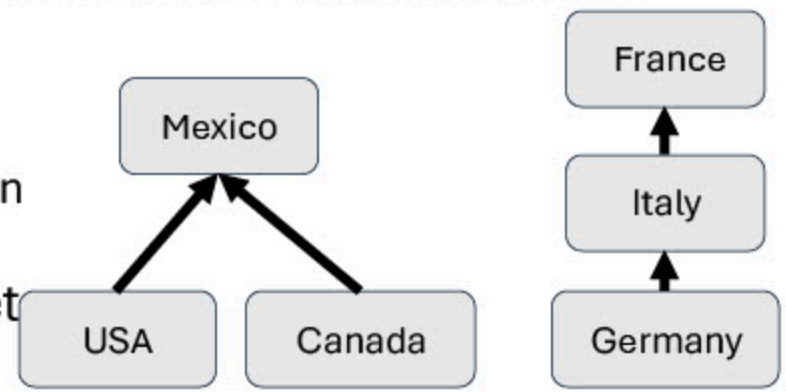
Given: a bunch of items, and some requirements (edges) for which pairs of items need to be in the same set

Goal: put the items into disjoint sets

Representing disjoint sets

- The goal of Union-Find is to result in a representation of disjoint sets:
  - Each item is a node in a graph
  - Each disjoint set is a tree within a *forest*
  - Each disjoint set has one element which is the *representative element*
    - That item is the root of the tree.

- The algorithm for finding / managing disjoint sets is called Union-Find
  - union: combine two sets
    - Do this when we find that those two sets have an element in common
  - find: find the representative element of a set
    - Given an element, find the root of the tree that it belongs to



Union-Find Summary

- The Union-Find algorithm finds disjoint sets
  - It groups nodes into sets such that, for each set:
    - The nodes in that set are connected to each other
    - None of the nodes in that set have an edge with a node outside that set

- Each node keeps track of:
  - Its data (the element)
  - Its parent (another node)
    - `node.parent == node` if node is a root
    - `node.parent == node2` if node2 is its parent
- Operations
  - `union(node1, node2)` combines the sets containing node1 and node2 into a single set
  - `find(node)` returns node's root

```
find (node):
    while (node.parent != node):
        node = node.parent
    return node

union(node1, node2):
  root1 = find(node1)
  root2 = find(node2)
  if root1 != root2:
    if root1.rank > root2.rank:
      root2.parent = root1
    else:
      root1.parent = root2
      if root1.rank == root2.rank:
        root2.rank++
```

Poll: A tree is a type of graph. What are the differences between a tree and a graph?

A. Each node in a tree must have exactly one parent (unless it is the root)

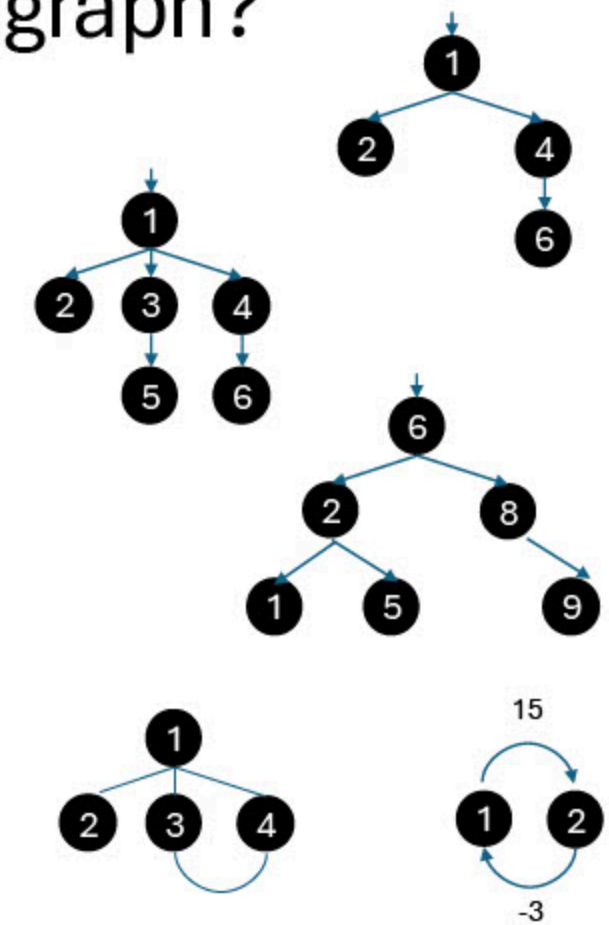
B. Each node in a tree must have at least one child

C. Trees are required to have at least one cycle

D. Trees are not allowed to have cycles

E. Edges in a tree must be weighted

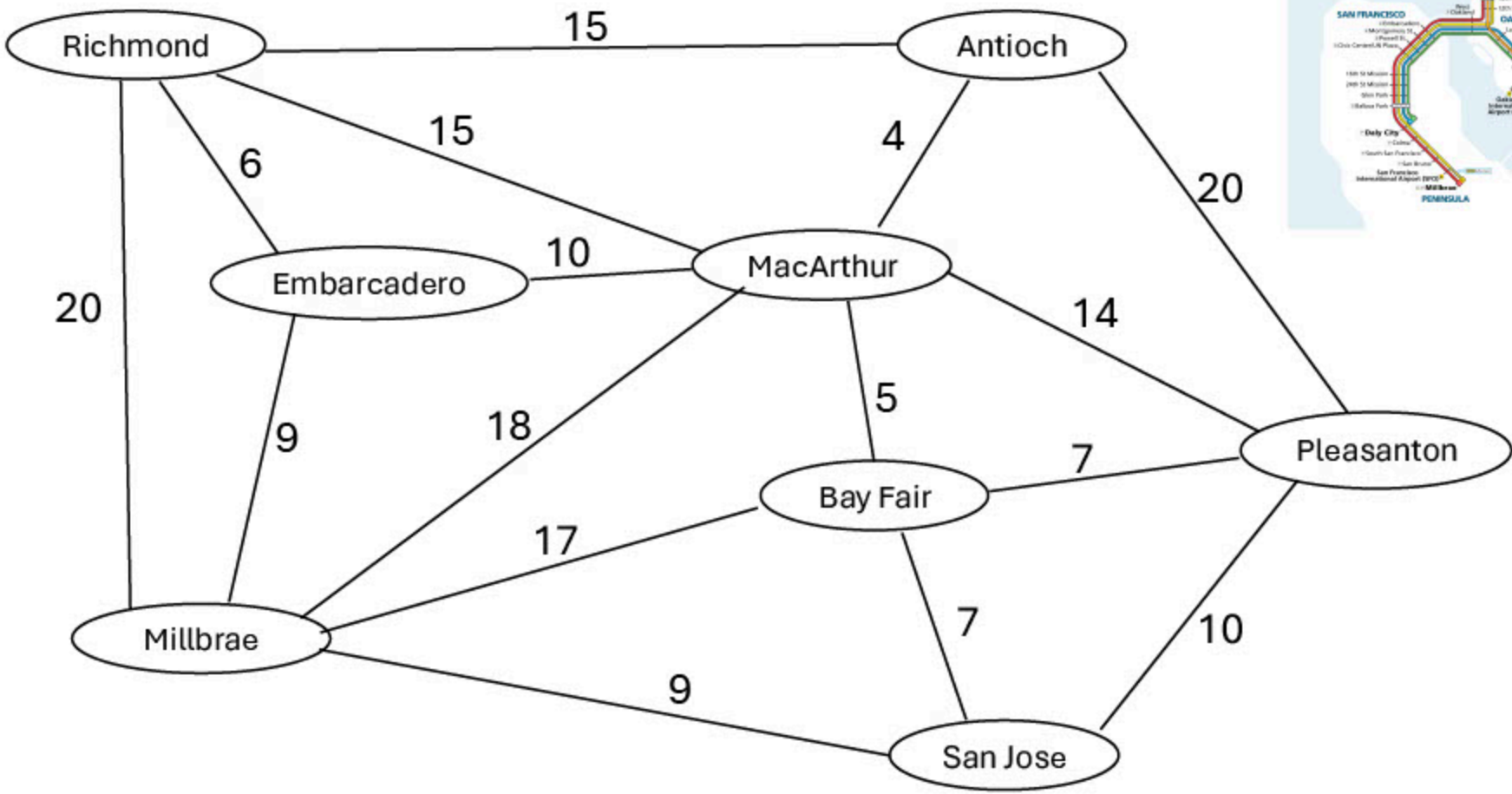
F. Edges in a tree cannot be weighted



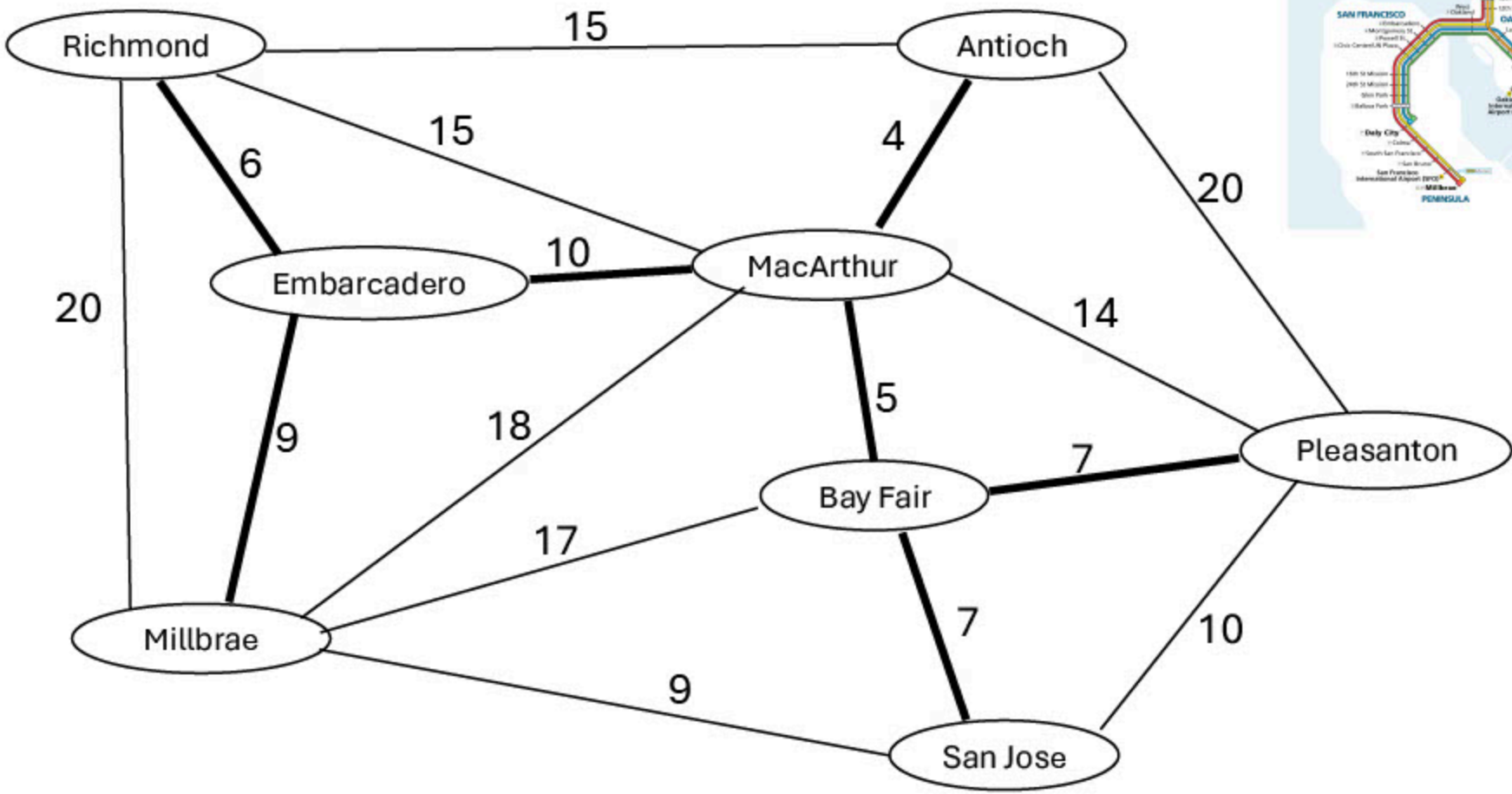
Let's say we're designing the BART system...

- Given:
  - Location of each station
  - Cost of building tracks from each station to each other station
- Goal:
  - Find the cheapest way to build tracks such that we can get from any station to any other station

https://www.bart.gov/system-map


Richmond
Antioch
Embarcadero
MacArthur
Bay Fair
Pleasanton
Millbrae
San Jose
15
15
6
4
20
10
20
14
9
18
5
7
17
7
10
9

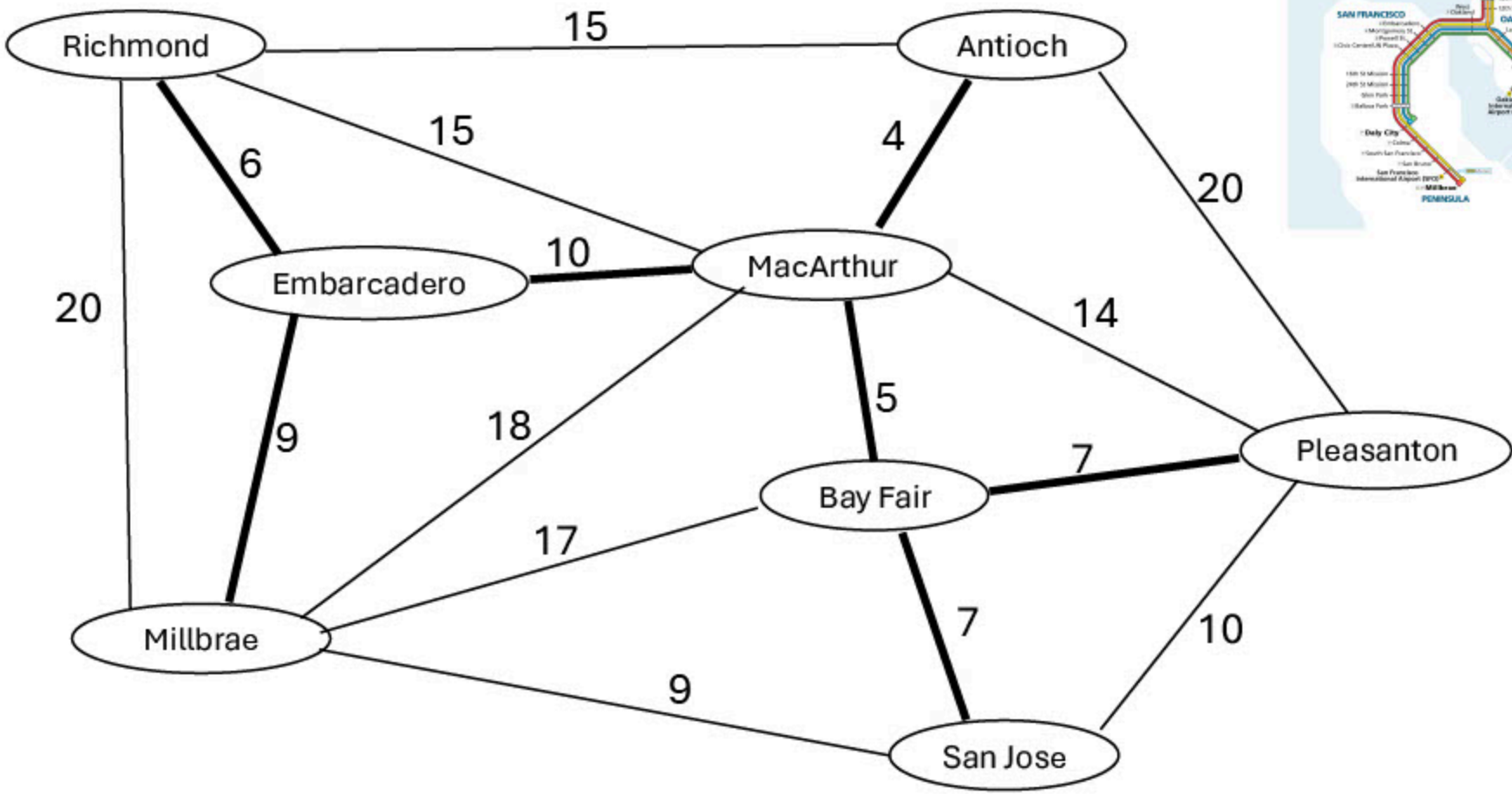
EAST BAY
SAN FRANCISCO
OAKLAND
PENINSULA
SAN JOSE


Richmond
Antioch
Embarcadero
MacArthur
Bay Fair
Pleasanton
Millbrae
San Jose
15
15
6
4
20
10
20
14
9
18
5
7
17
7
10
9

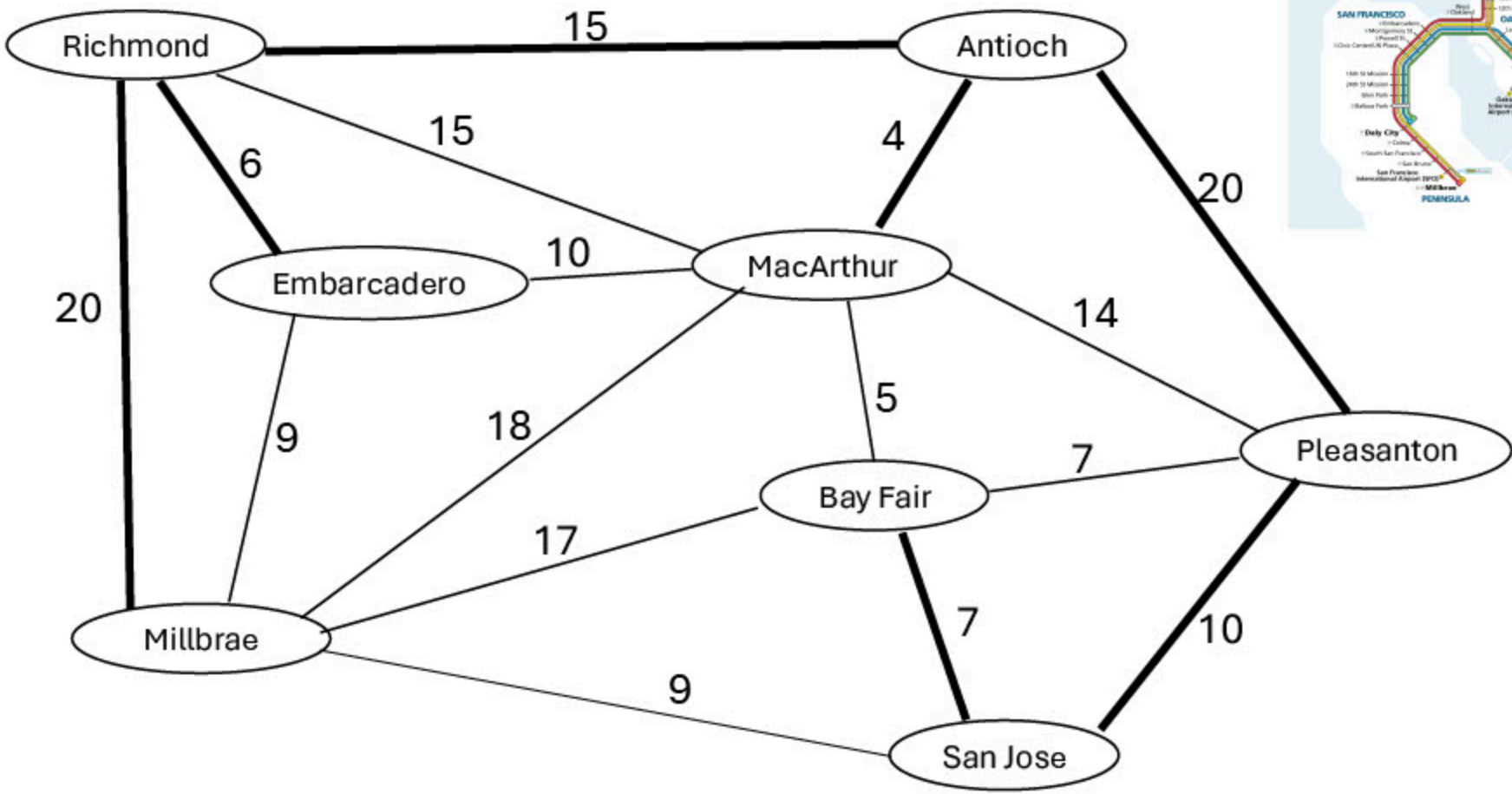
EAST BAY
SAN FRANCISCO
OAKLAND
PENINSULA
SAN JOSE

Spanning Tree

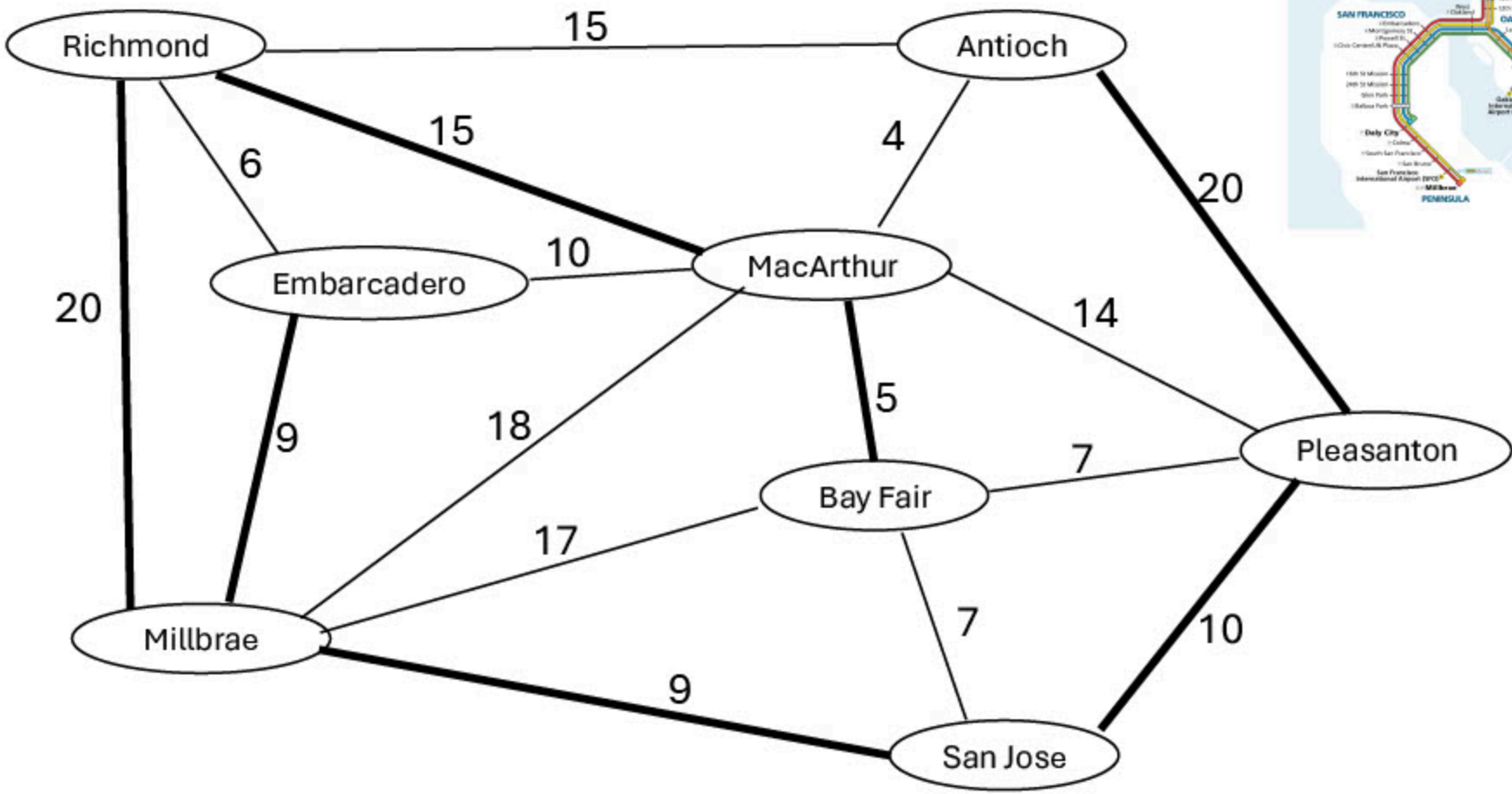
A spanning tree of a graph G is any tree that contains all the vertices in G


Richmond
Antioch
Embarcadero
MacArthur
Bay Fair
Pleasanton
Millbrae
San Jose
15
15
6
4
20
10
20
14
9
18
5
7
17
7
10
9

EAST BAY
SAN FRANCISCO
OAKLAND
PENINSULA
SAN JOSE


Richmond
Antioch
Embarcadero
MacArthur
Pleasanton
Bay Fair
Millbrae
San Jose
15
15
6
4
20
10
20
14
9
18
5
7
17
7
10
9

EAST BAY
SAN FRANCISCO
OAKLAND
PENINSULA
SAN JOSE


Richmond
Antioch
Embarcadero
MacArthur
Bay Fair
Pleasanton
Millbrae
San Jose
15
15
6
4
20
10
20
14
9
18
5
7
17
7
10
9

EAST BAY
SAN FRANCISCO
OAKLAND
PENINSULA
SAN JOSE

Minimum Spanning Tree

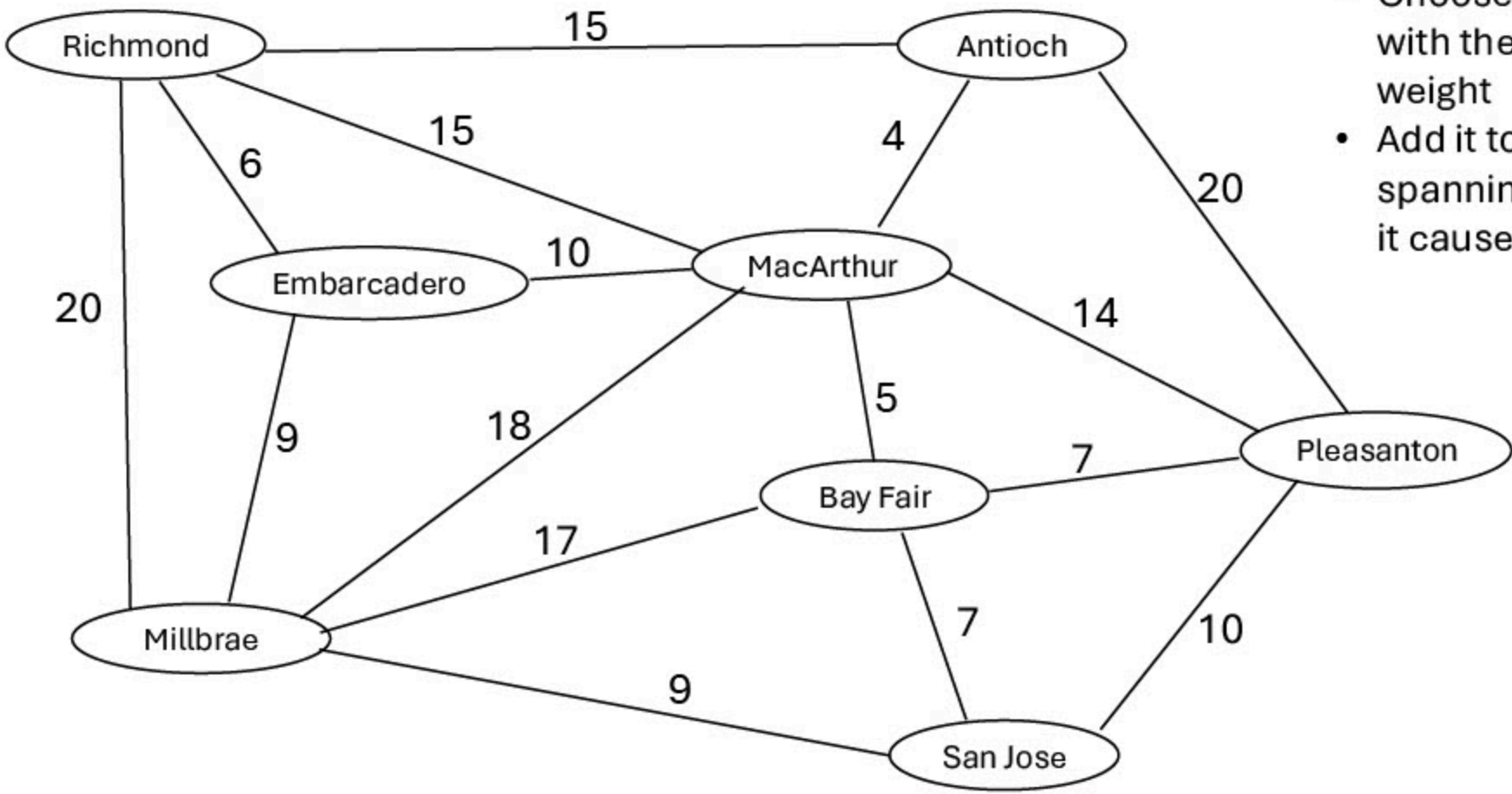
An MST of a graph G is a tree that connects all the vertices of G with the minimum possible edge weight sum

Kruskal's Algorithm

Until all the vertices are connected:

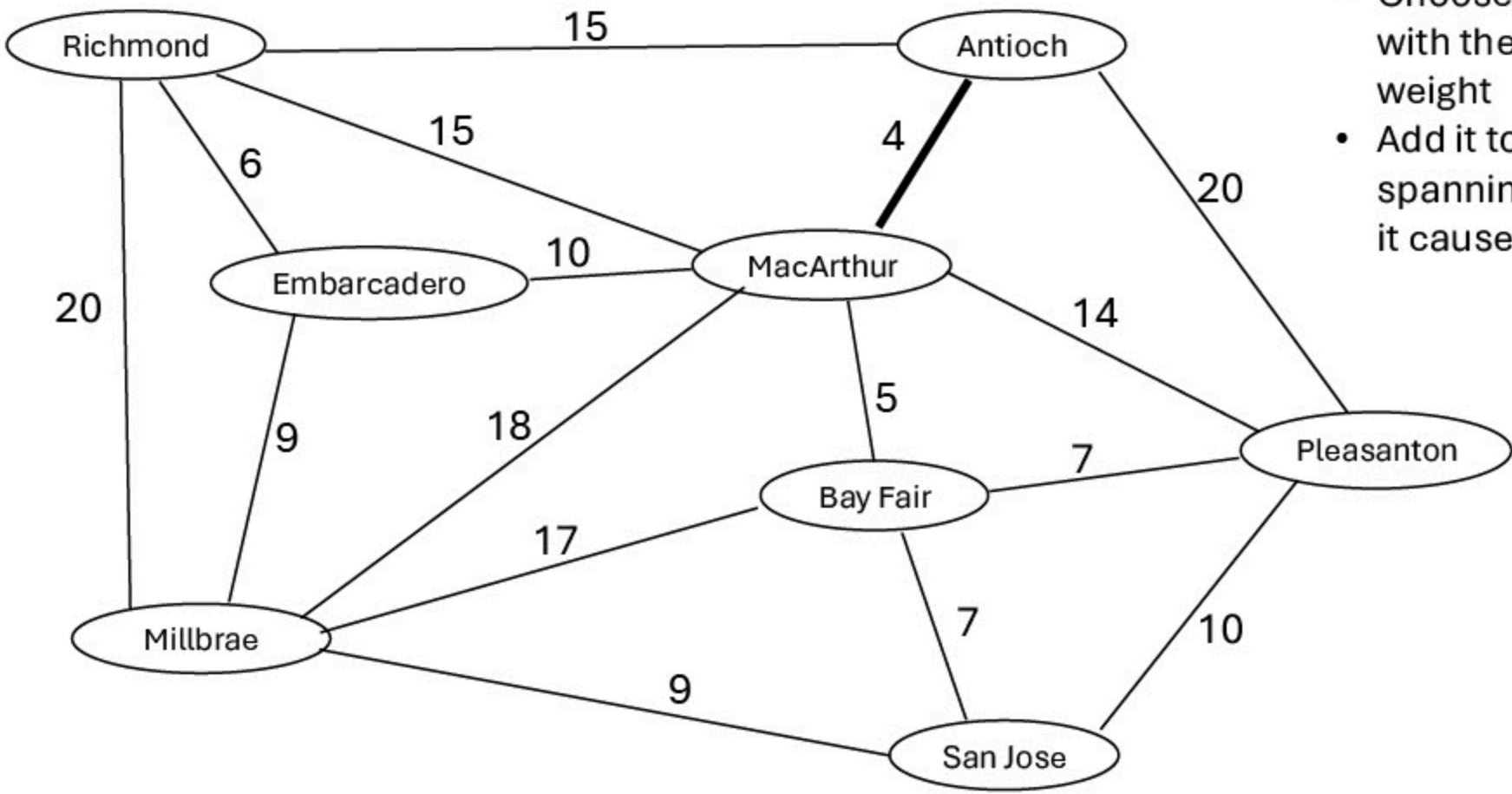
- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle

This algorithm is greedy and optimal for finding MSTs.



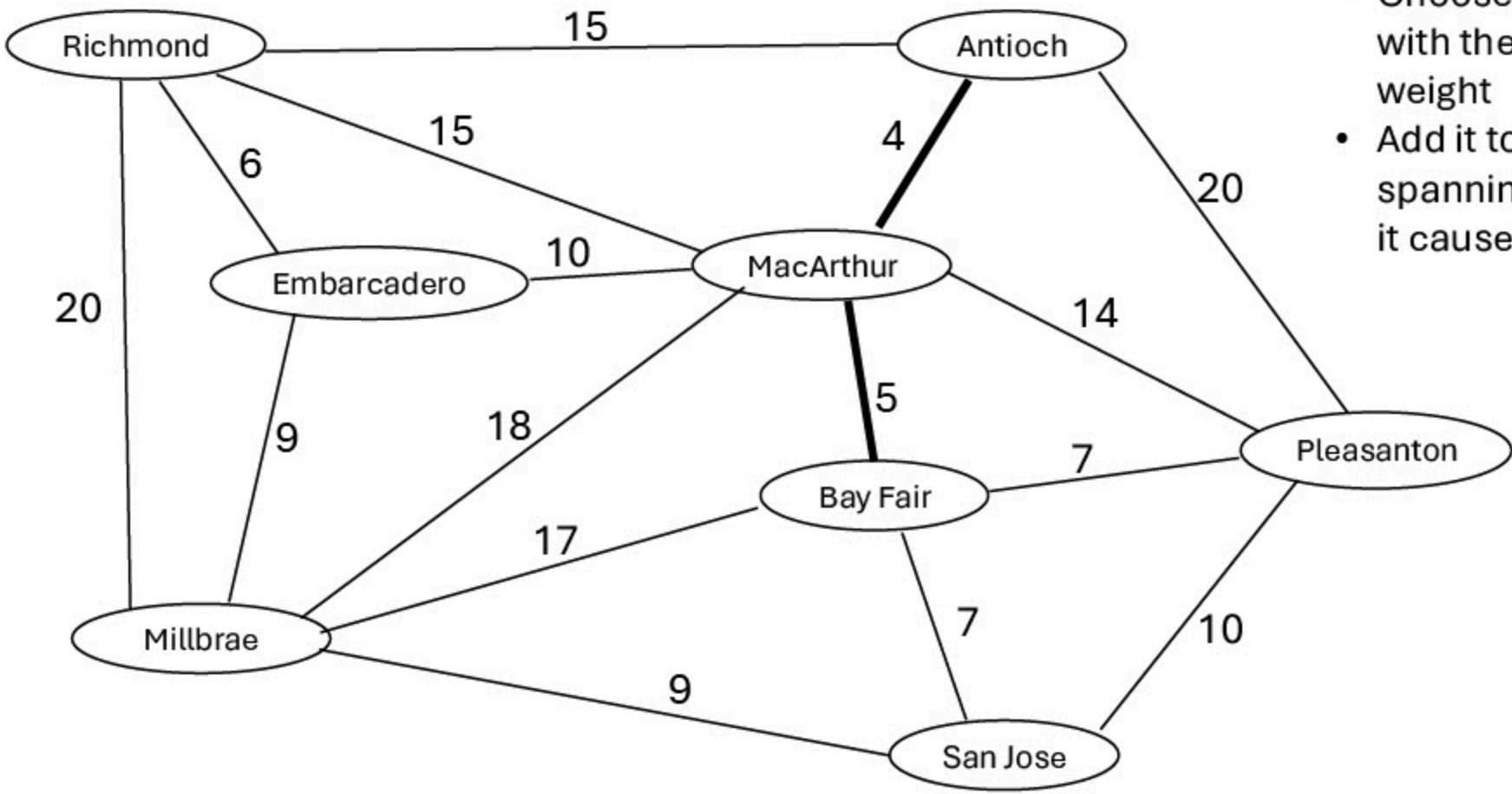
Until all the vertices are connected:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



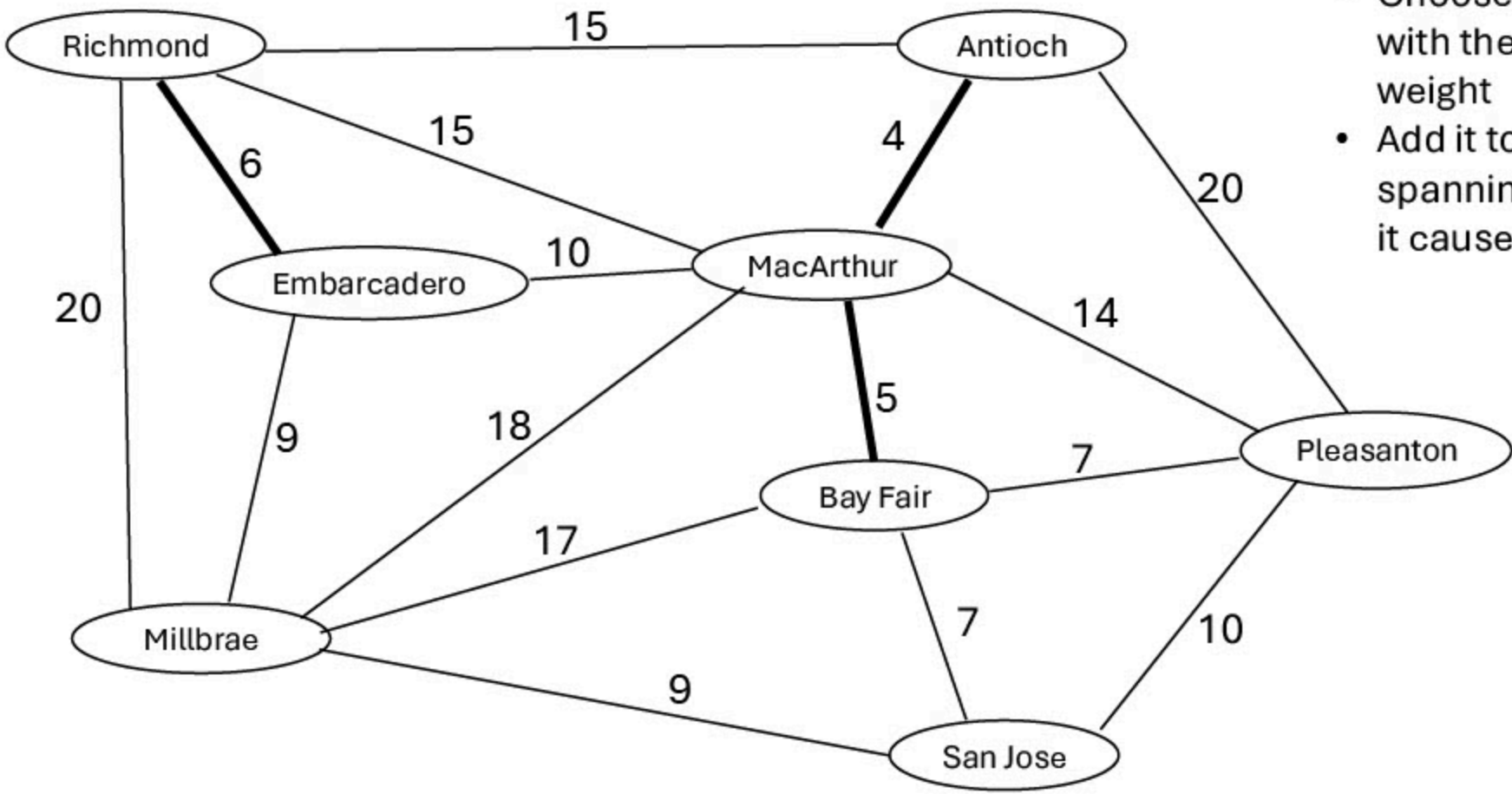
Until all the vertices are connected:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



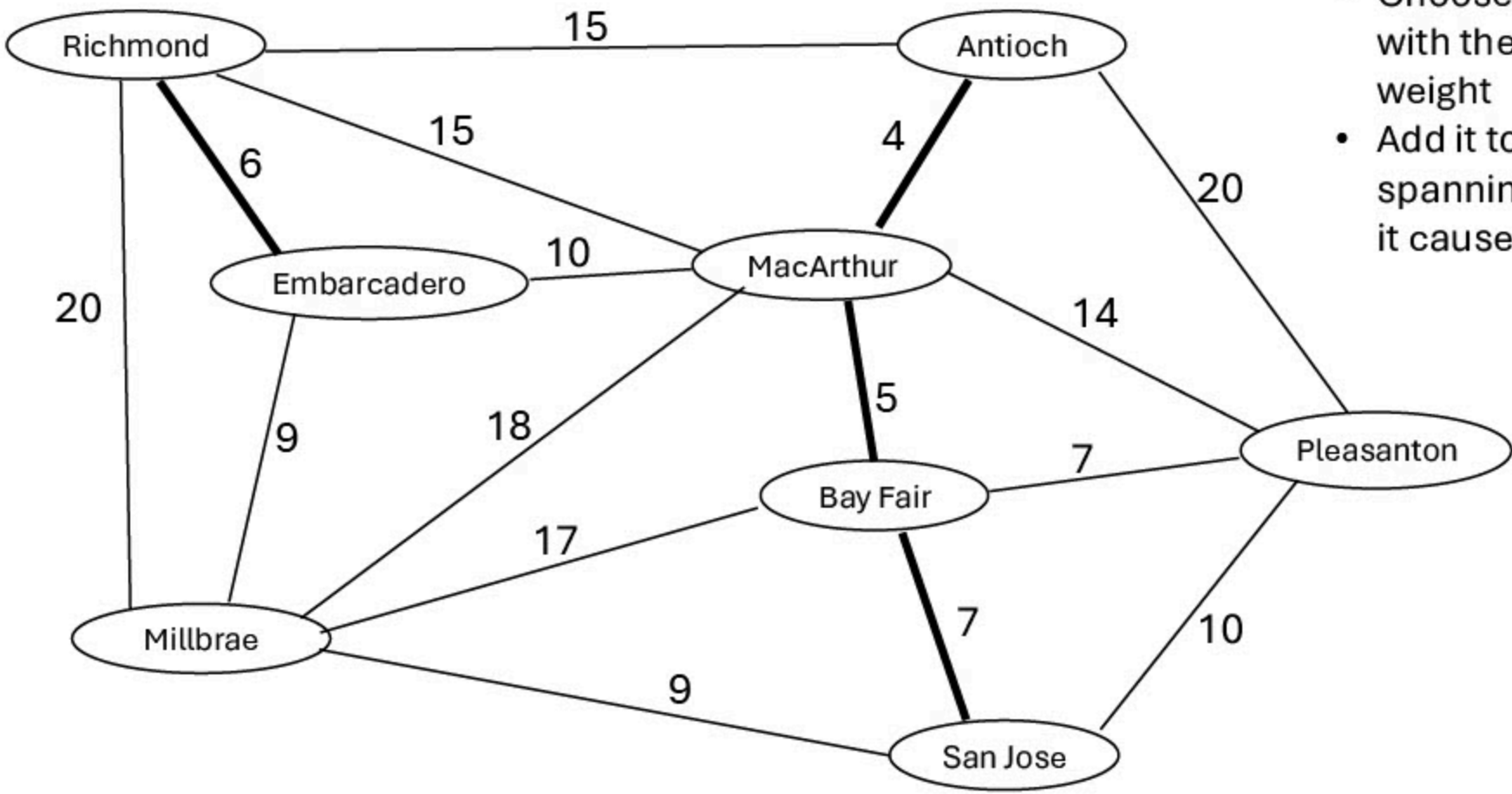
Until all the vertices are connected:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



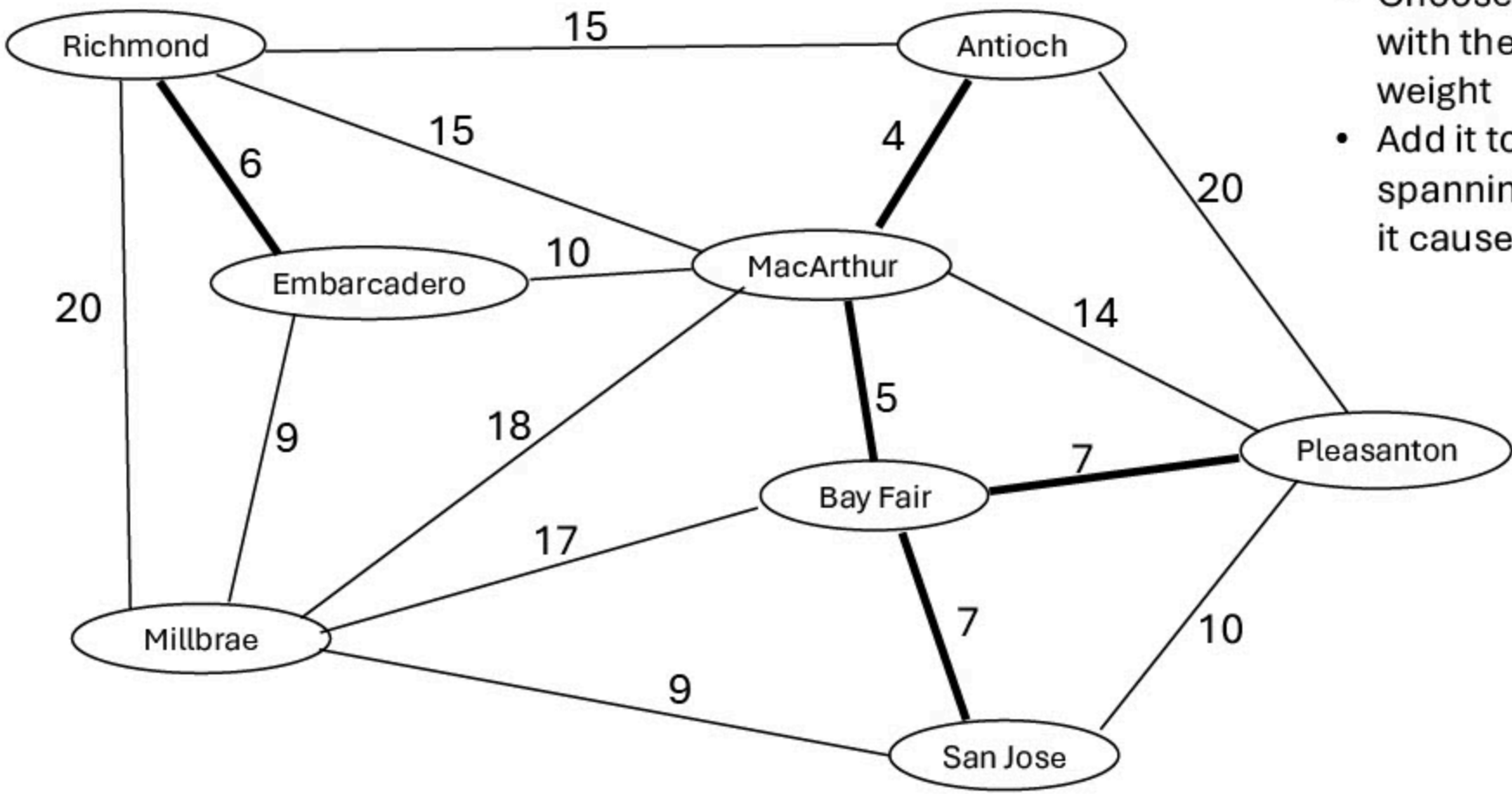
Until all the vertices are connected:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



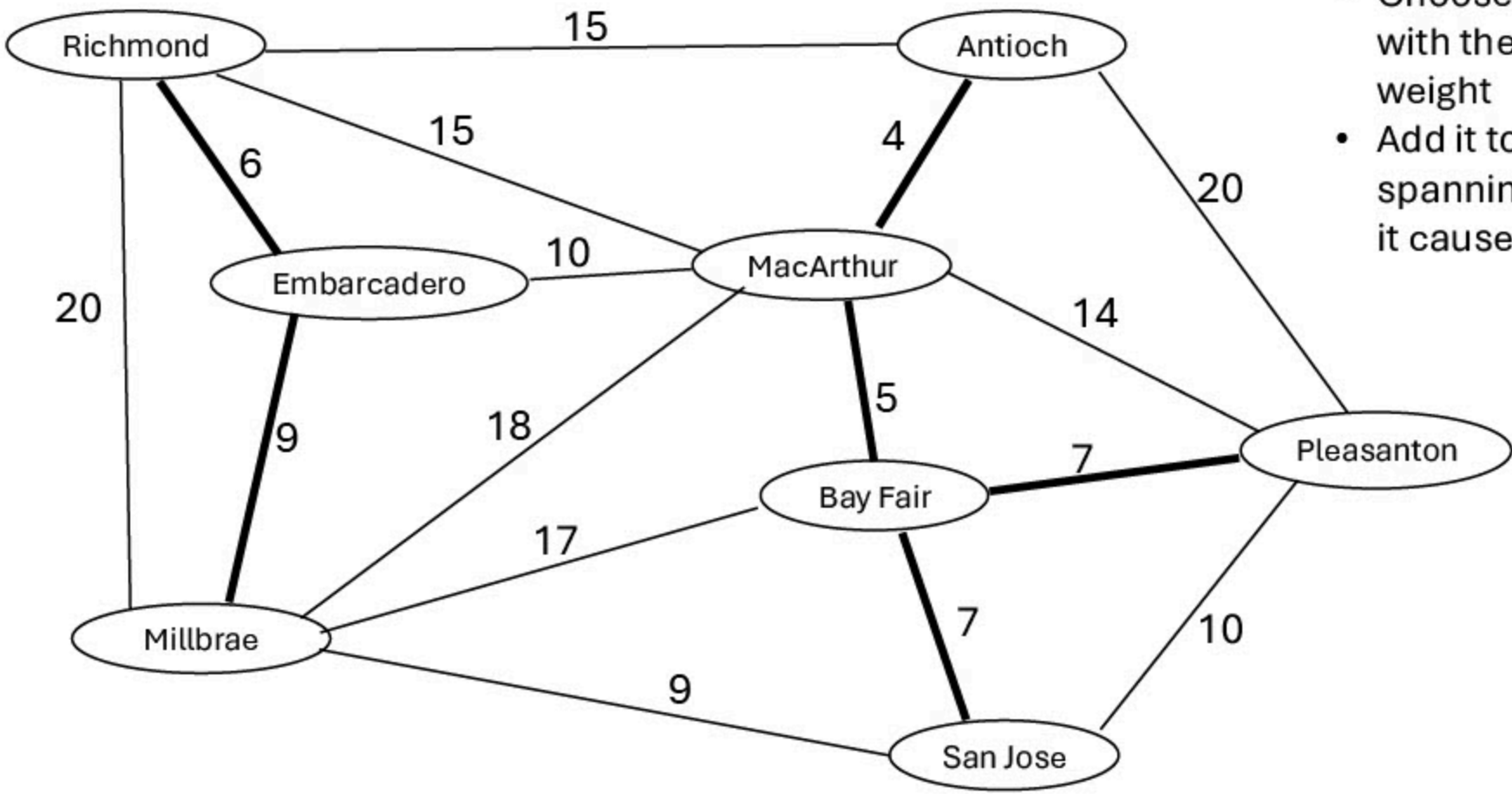
Until all the vertices are connected:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



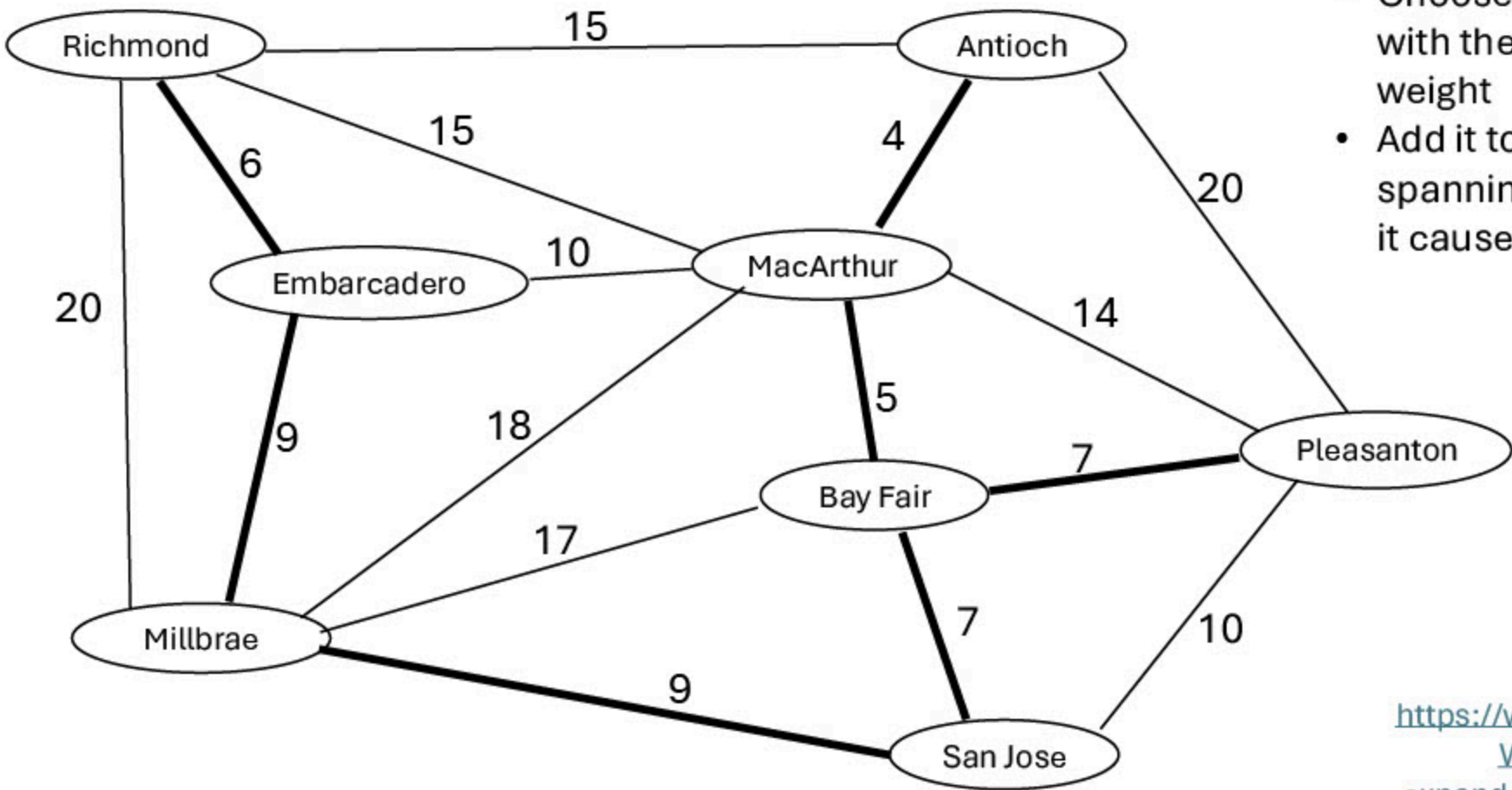
Until all the vertices are connected:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



Until all the vertices are connected:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



Until all the vertices are connected:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle

https://www.quora.com/Why-hasnt-BART-expanded-to-the-South-Bay

Poll: How can we get the edge with the smallest weight at each step?

1. Store the edges in a sorted list, and remove the first item one by one
2. Store the edges in a regular (non-sorted) list, and search for and remove the smallest item one by one
3. Store the edges in a set, and remove items one by one
4. Store the edges in a hash table, and remove the first item one by one

Poll: How can we tell if an edge (between `node1` and `node2`) would cause a cycle?

1. Union-find: if the `find` operation returns the same root for both `node1` and `node2`, then that edge would cause a cycle. Otherwise, it wouldn't.
2. Union-find: if the find operation for `node1` returns `node2`, or vice versa, then that edge would cause a cycle. Otherwise, it wouldn't.
3. Recursive backtracking: starting at `node1`, search for all nodes that can be reached. If `node2` is in that set, then that edge would cause a cycle. Otherwise, it wouldn't.

Kruskal's Algorithm (formal)

1. Create a forest F (a set of trees), where each vertex in the graph is a separate tree.

2. Create a sorted list S containing all the edges in the graph.

3. While S is non-empty and F is not a spanning tree,

    i. Remove the edge with the minimum weight from S.

    ii. If the removed edge connects two different trees, then add it to the forest F, combining two trees into a single tree.

https://en.wikipedia.org/wiki/Kruskal's_algorithm

Kruskal's Algorithm (formal, but applied to public transit)

1. Create a forest F (a set of trees), where each vertex in the graph is a separate tree.

Create a union-find node for each station, where the data is the station and the parent is itself (root)

2. Create a sorted list S containing all the edges in the graph.

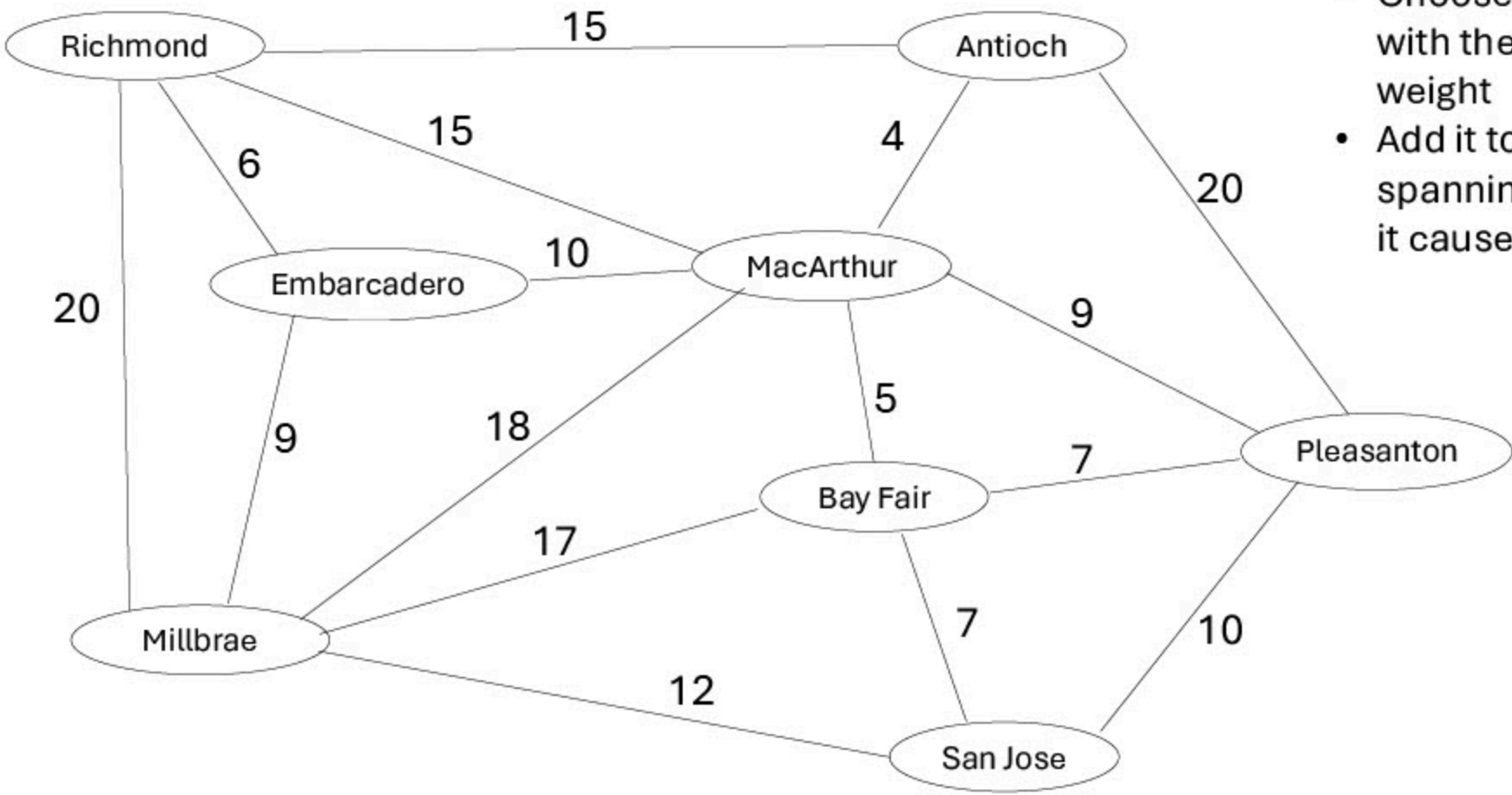
Create a graph edge between possible each pair of stations, and put all these edges in a sorted list

3. While S is non-empty and F is not a spanning tree

There are some stations `s1` and `s2` such that the train cannot get from `s1` to `s2`
The Union-Find still has multiple trees in it

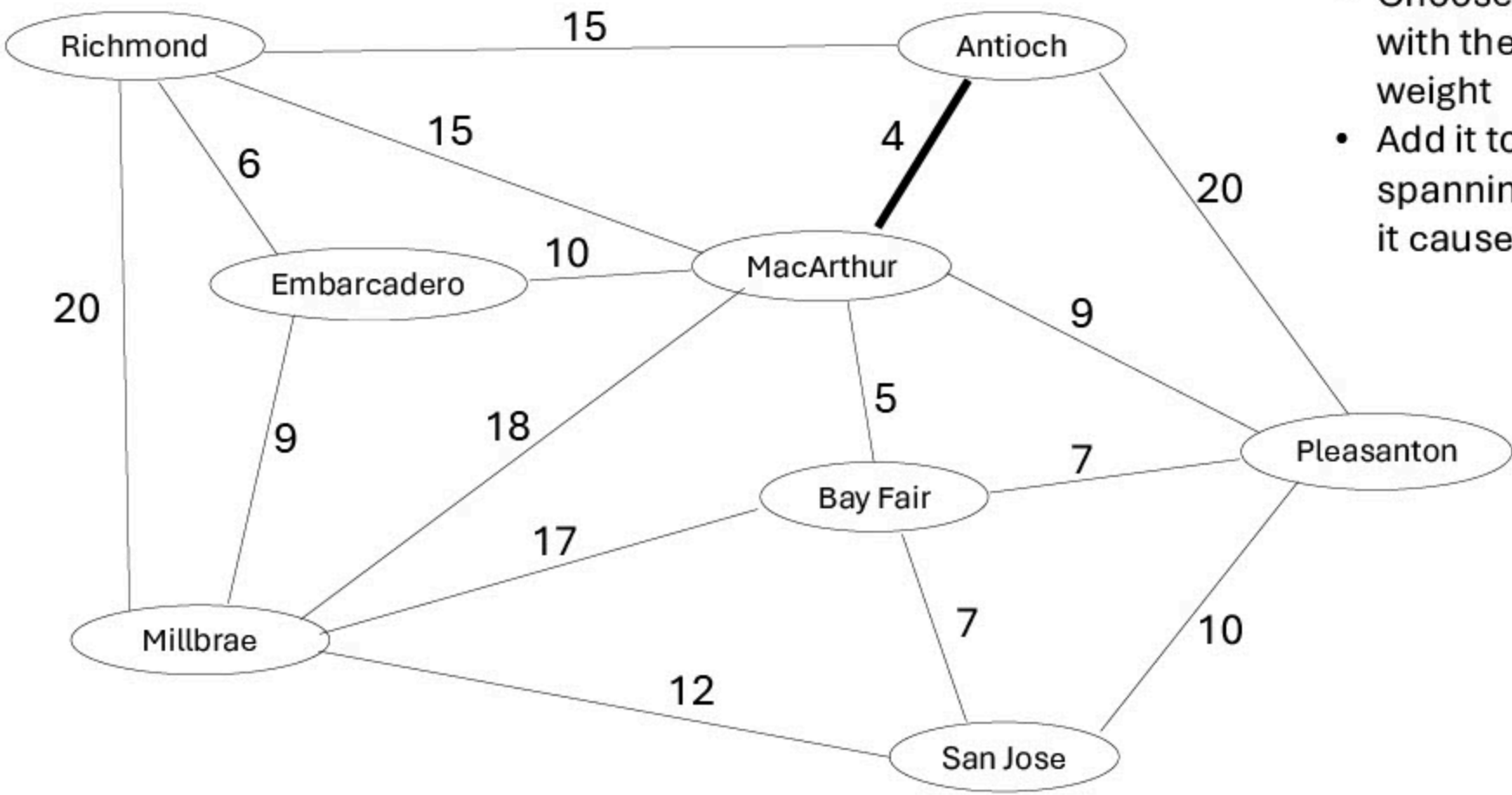
Another Example MST

(different weights)



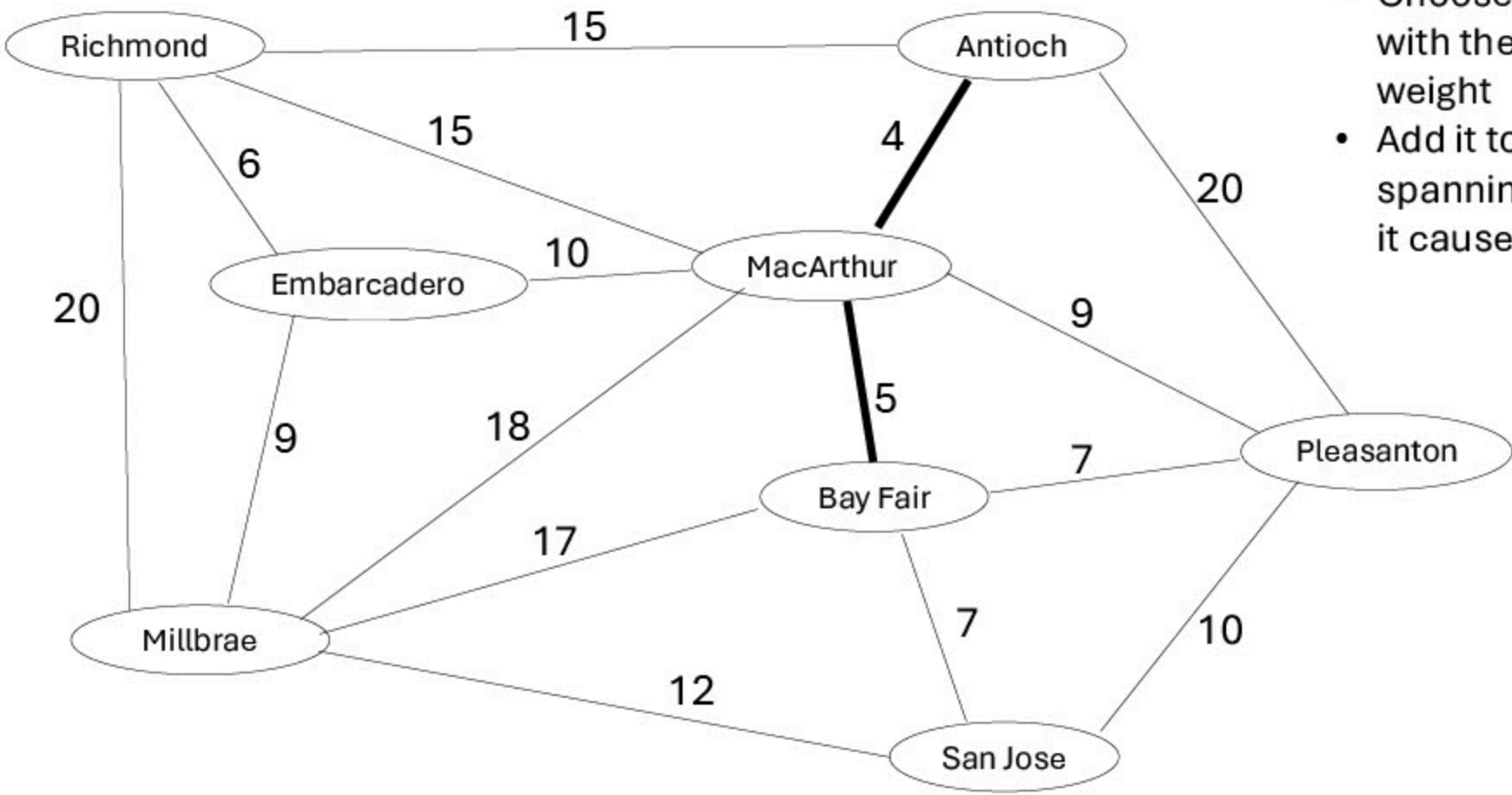
Until all the vertices are connected together:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



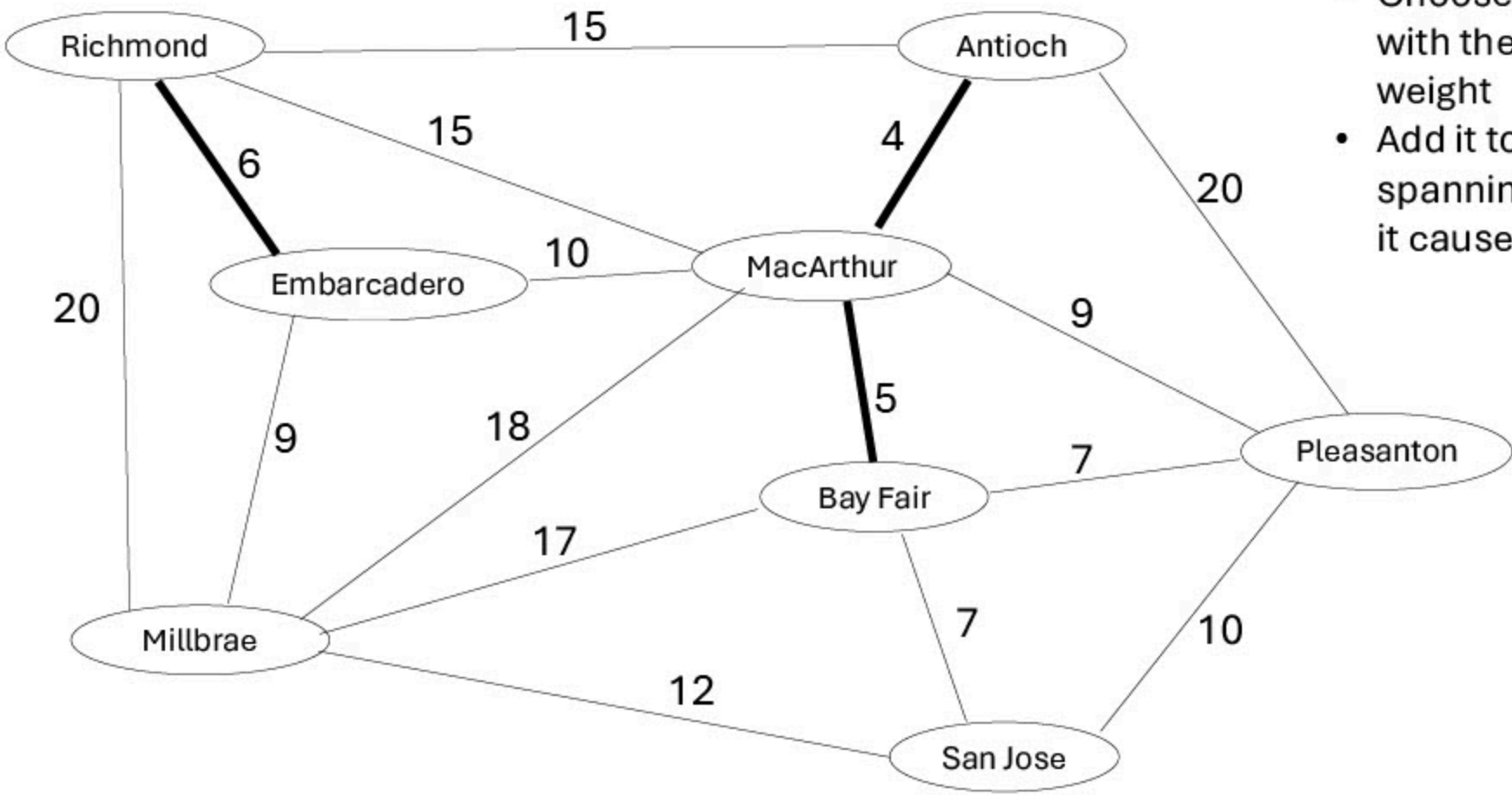
Until all the vertices are connected together:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



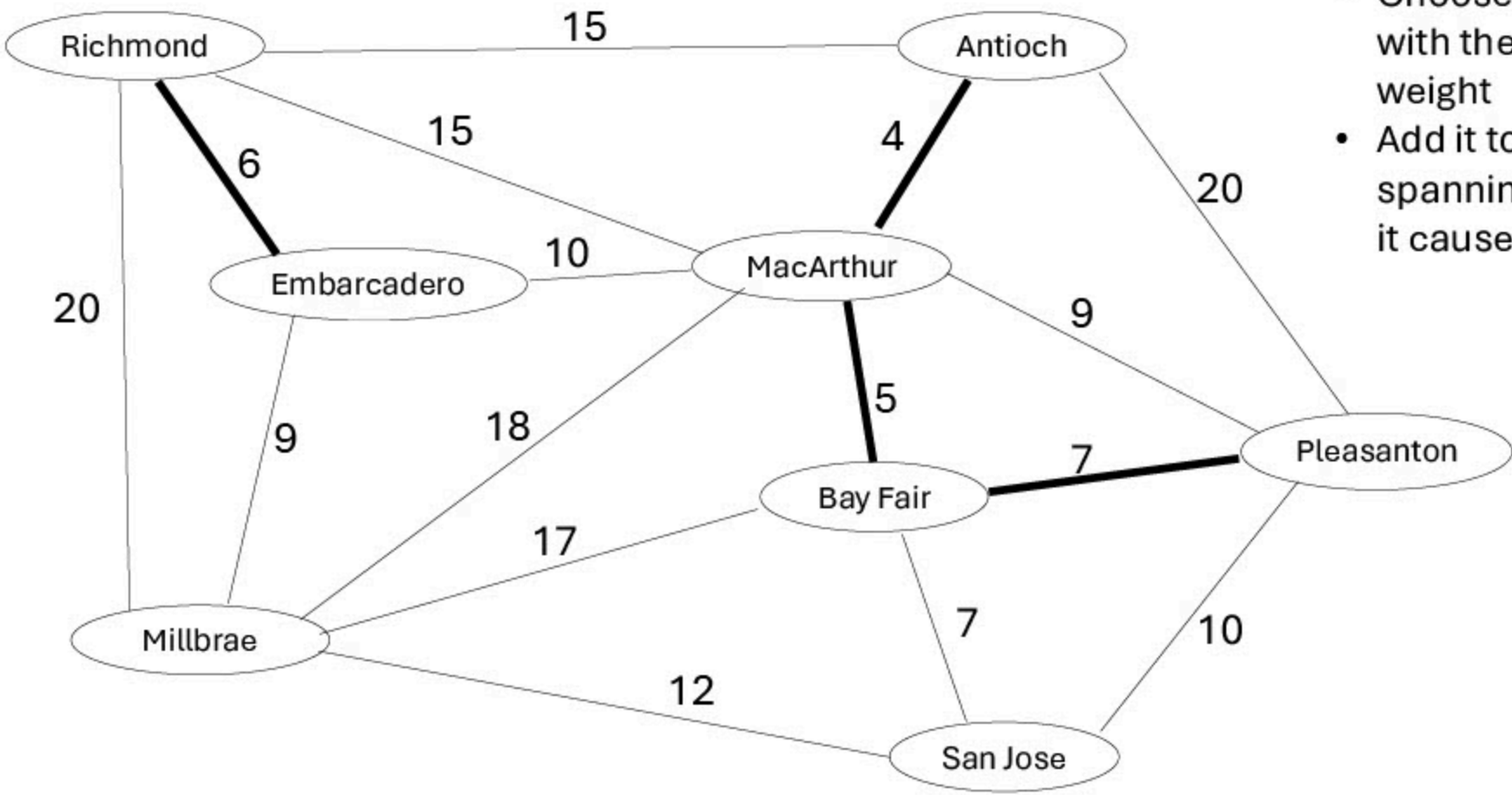
Until all the vertices are connected together:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



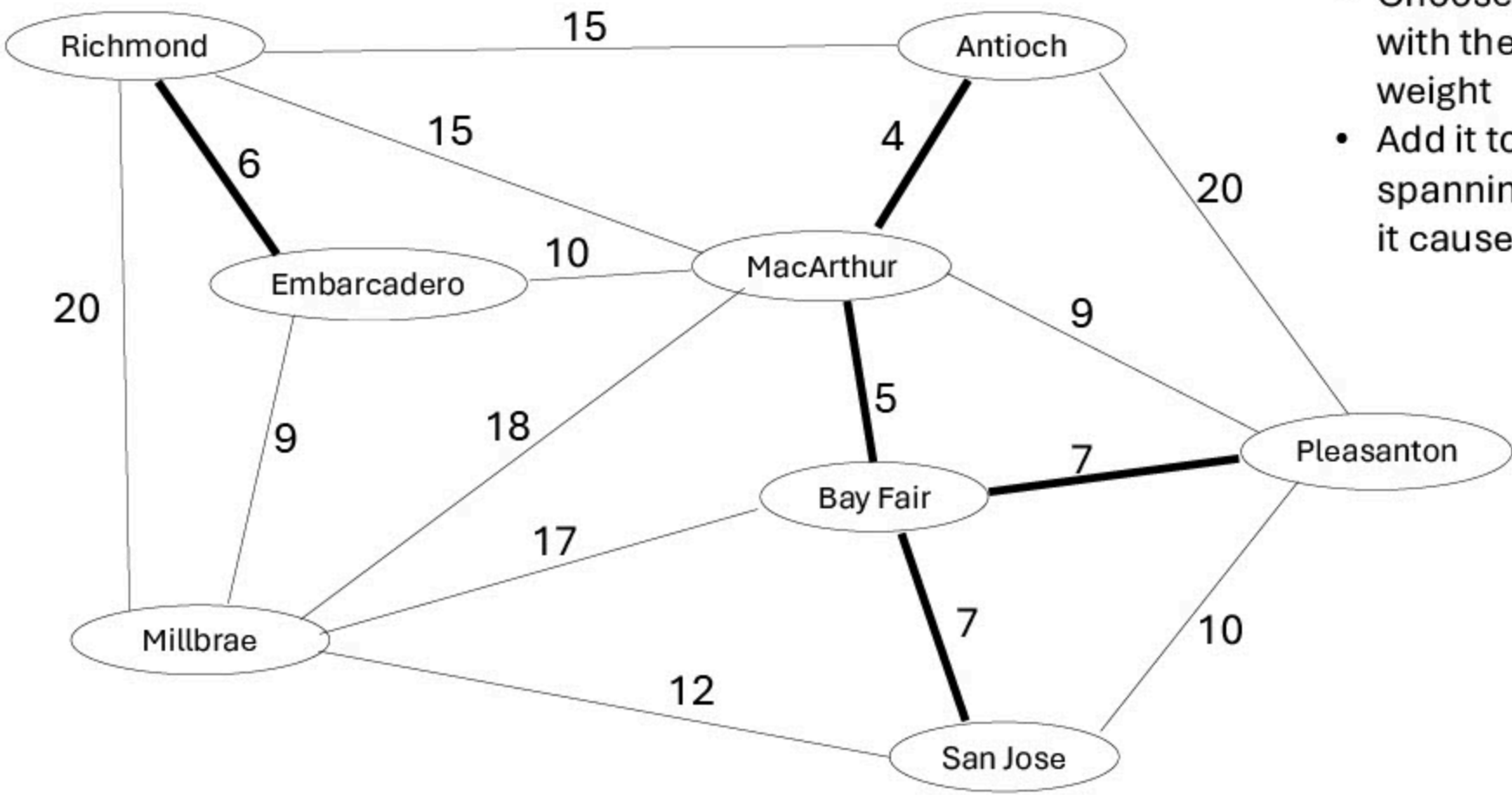
Until all the vertices are connected together:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



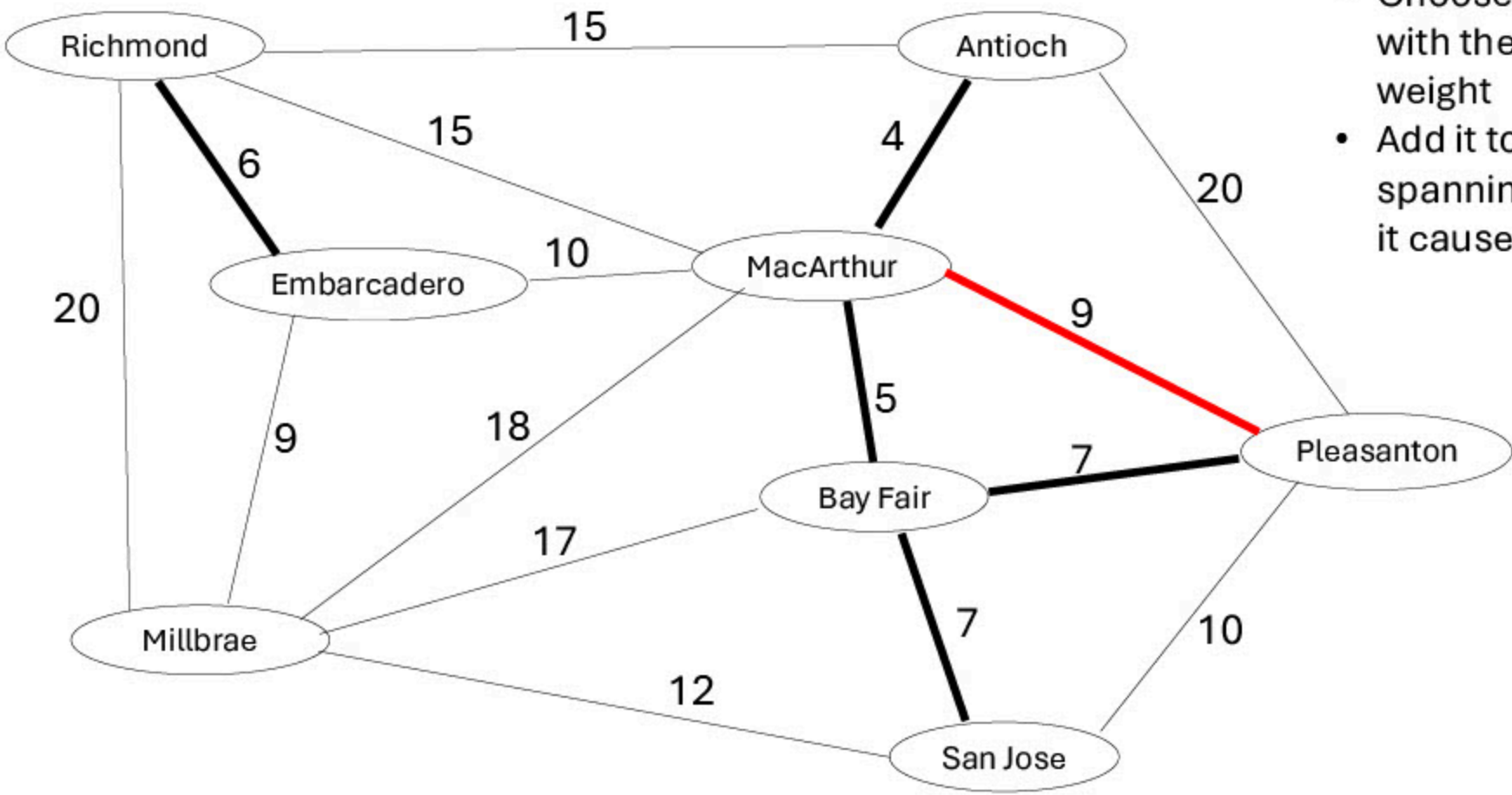
Until all the vertices are connected together:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



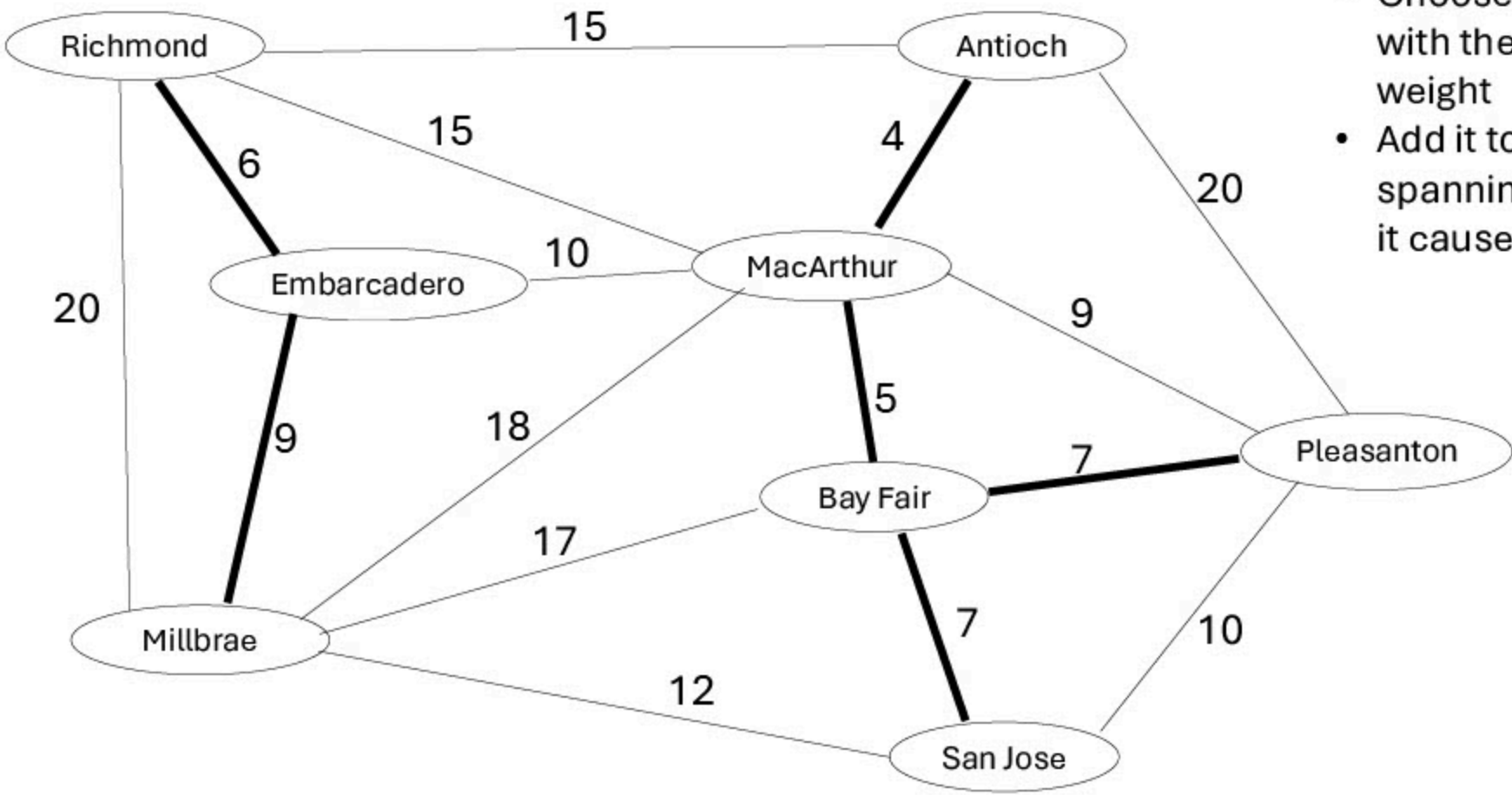
Until all the vertices are connected together:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



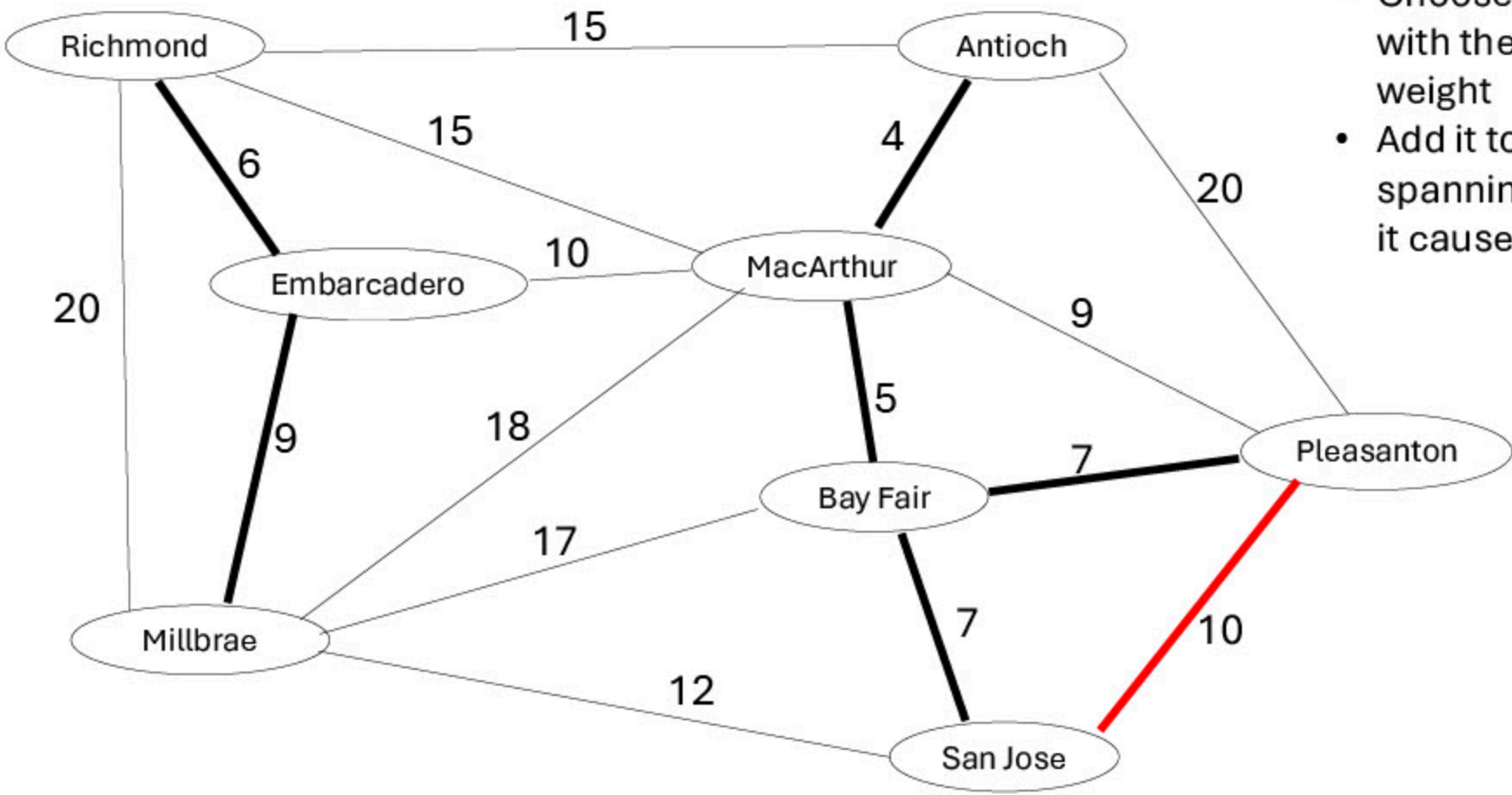
Until all the vertices are connected together:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



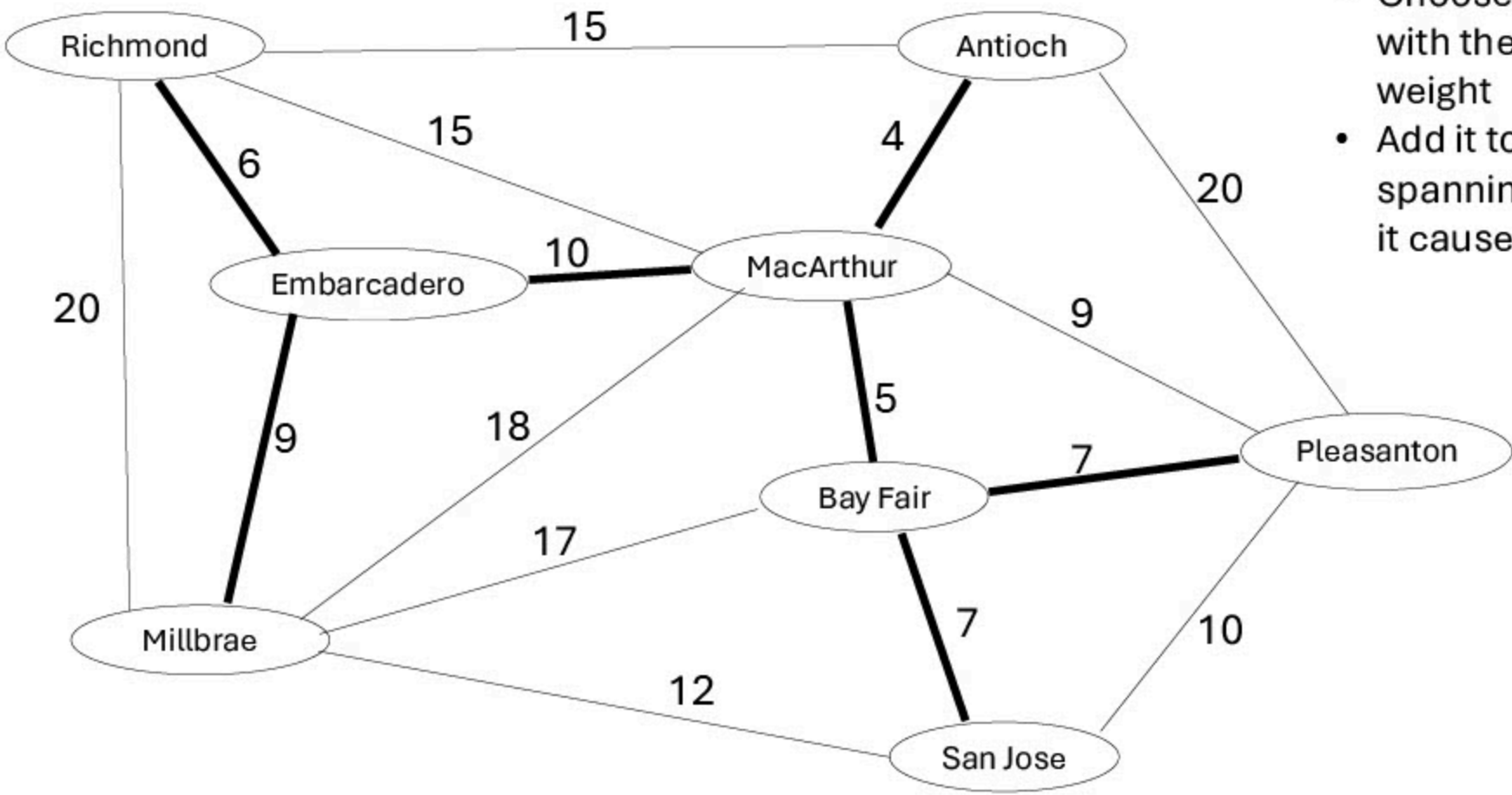
Until all the vertices are connected together:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



Until all the vertices are connected together:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle



Until all the vertices are connected together:

- Choose the edge with the minimum weight
- Add it to the spanning tree unless it causes a cycle

Poll: Which of the following are true?

1. To iterate through the edges, smallest to largest, we can store them in a sorted list
2. To determine whether adding an edge would cause a cycle, we can use union-find to see if they are already in the same tree
3. We complete Kruskal's Algorithm when each node has at least one edge connected to it in the tree
4. The weighted graph edges chosen for the MST are different from the directed parent edges used for union-find

Poll:

1. What is your main takeaway from today?

2. What would you like to revisit next time?